

Warsaw University of Technology

FACULTY OF
ELECTRONICS AND INFORMATION TECHNOLOGY



Institute of Computer Science

Master's diploma thesis

in the field of study Computer Science
and specialisation Computer Science

AI-based profanity recognition and rephrasal system for multiplayer video
game chats

Bukowski Damian

student record book number 303828

thesis supervisor

Tomasz Martyn, PhD, DSc

WARSAW 2026

Keywords: NLP, Video Games, C++, BERT

Słowa kluczowe: NLP, Gry Komputerowe, C++, BERT

Contents

1. Theoretical Background	9
1.1. The problem: toxicity as text classification	9
1.2. Distributional word representations	9
1.3. Convolutional networks over text	10
1.4. The Transformer and contextual encoders	11
1.5. Knowledge distillation and DistilBERT	11
1.6. Subword tokenization and the parity requirement	12
1.7. Sequence labelling and toxic span detection	12
1.8. Evaluation	13
1.9. Bias and fairness in toxicity models	13
1.10. Summary	14
2. Language and Runtime: Why Mostly C++ and a Little Python	14
2.1. The governing constraint: per-message latency	14
2.2. Runtime independence from Python	15
2.3. The build-time / run-time boundary	15
2.4. The parity problem: where the boundary can break	16
2.5. Honest trade-offs	16
2.6. Summary	17
3. Results	17
3.1. Experimental setup	17
3.2. Evaluation methodology	18
3.3. Baselines	18
3.4. Component results	19
3.5. End-to-end results	19
3.6. Runtime performance	20
3.7. Ablations	20
3.8. Error analysis	21
3.9. Adversarial robustness	21
3.10. Summary	22
References	23
List of Symbols and Abbreviations	24
List of Figures	24

1. Theoretical Background

This chapter sets out the ideas the rest of the thesis builds on. It moves from the general problem of classifying text, through the two families of neural architecture the system uses – convolutional networks over word embeddings and the Transformer encoder – and then to the smaller, more specific machinery the pipeline depends on: subword tokenization, knowledge distillation, sequence labelling for span detection, the metrics used to judge all of it, and the fairness concerns that any deployed toxicity filter has to answer for. While not novel on its own, this thesis aims to combine them and implement in a way that has not been done before.

1.1. The problem: toxicity as text classification

At the coarsest level, filtering game chat is a text-classification problem. Given a short message, we want a decision – is this toxic or not – and, if it is, we want to know which part of it carries the toxicity so that it can be removed rather than the whole message discarded. These are two different tasks. The first is message-level classification; the second is token- or span-level labelling. The system in this thesis uses one model for each, and much of the design tension described in Chapter ?? comes from the fact that a message-level verdict and a span-level edit live at different granularities.

Toxicity is also not a single thing. The Jigsaw dataset that trains the message-level model treats the primary toxicity score as continuous and attaches a set of finer sub-attributes – obscenity, insult, threat, identity attack and so on **jigsaw2019unintended**. Video game chats also add their own caveat: much of the hostility that players actually experience contains no profanity at all. Phrases, such as: “uninstall the game” and “you are actively throwing” are hostile without a single dictionary swear word in them. A filter that only removes profane tokens will pass most real harassment straight through. This is why the thesis distinguishes, throughout, between lexical profanity and directed hostility, and why span removal – deleting a hostile clause, not just a rude word – is the target capability.

1.2. Distributional word representations

Neural text models do not consume characters or words directly; they consume vectors. The foundational idea is the distributional hypothesis: words that appear in similar contexts tend to have similar meanings, so a word can be represented by a dense vector learned from the company it keeps. Two families of pre-trained embeddings are relevant here. **word2vec** learns vectors by predicting a word from its context, or the reverse, with a shallow network trained over very large corpora **mikolov2013word2vec**. GloVe instead factorizes a global word co-occurrence matrix, so that vector differences encode ratios of co-occurrence probabilities **pennington2014glove**. Both produce 300-dimensional

vectors in their common public releases, and both are usable as the embedding layer of a downstream classifier.

Two properties of these embeddings matter for the design. First, they can be either frozen (used as fixed features) or fine-tuned (updated by the downstream task’s gradient). A study of CNN text classifiers found that allowing the embeddings to adapt generally helps, and the message-level model here follows that recommendation **kim2014cnn**. Second, embeddings are word-level, so they inherit a vocabulary, meaning that a word the pre-trained model never saw has no vector. This out-of-vocabulary problem is acute for game chat, whose core lexicon – noob, gg, wp, mid, feed – barely overlaps with the news-and-comment corpora on which public embeddings are trained. Chapter 3 returns to this as one of the measured limitations of the message-level model.

1.3. Convolutional networks over text

The message-level model is a convolutional neural network, based on a similar network created for Convolutional Neural Networks for Sentence Classification by Yoon Kim. **kim2014cnn**. The intuition and hope is that a toxic message is usually toxic because of a short, position-independent patternsuch as a slur, an obscene phrase, an insulting construction. I believe that a one-dimensional convolution will work very well as a detector for such local patterns.

A message of L tokens becomes a matrix $\mathbf{X} \in \mathbb{R}^{L \times d}$ by stacking the d -dimensional embedding of each token. A filter $\mathbf{w} \in \mathbb{R}^{h \times d}$ of window size h slides over the token axis, and at each position i produces a feature

$$c_i = f(\mathbf{w} \cdot \mathbf{X}_{i:i+h-1} + b), \quad (1)$$

where f is a non-linear function and b a bias. Sweeping the filter over the whole message yields a feature map $\mathbf{c} = (c_1, \dots, c_{L-h+1})$. Lastly, a max-over-time pooling is applied over the feature map. This produces a maximal value $\hat{c} = \max c$, also being the most important feature, that corresponds to a specific filter **kim2014cnn**. Max pooling well-suited for the use-case in this thesis, since it doesn’t dilute a single toxic signal over a long message, which a method like average pooling would do.

Using several window sizes in parallel lets the network detect patterns of different lengths at once; the pooled outputs are concatenated into one feature vector and passed to a classifier head. Kim used windows of 3, 4 and 5 with 100 filters each. Zhang and Wallace’s follow-up sensitivity analysis studied how these choices interact and gave practitioners concrete guidance on filter widths, filter counts, activation and regularization **zhang2015sensitivity**: the wider window set used in this thesis (2, 3, 4, 5, 7) follows their observation that the optimal region size grows with document length, and Jigsaw comments are far longer than the single sentences in Kim’s benchmarks. Regularization in this family is provided by dropout on the pooled vector **srivastava2014dropout** and, in

Kim’s recipe, an ℓ_2 norm constraint on the classifier weights, both of which Chapter ?? adopts.

1.4. The Transformer and contextual encoders

Convolutional and recurrent models read text through a fixed local window or a sequential state. The Transformer replaced both with self-attention, a mechanism in which every token attends directly to every other token in the sequence, so that context of arbitrary distance is available in a single layer **vaswani2017attention**. Scaled dot-product attention computes, for queries $\mathbf{Q}$, keys $\mathbf{K}$ and values $\mathbf{V}$,

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \quad (2)$$

where d_k is the key dimension and the $\sqrt{d_k}$ scaling keeps the softmax in a well-conditioned range **vaswani2017attention**. Several such attention “heads” run in parallel and are combined, giving the model multiple views of the token-to-token relationships.

BERT applied a stack of Transformer encoder layers to language understanding, pre-training the network on two self-supervised objectives – masked-language modelling, in which random tokens are hidden and predicted from their bidirectional context, and next-sentence prediction – so that a single pre-trained model could be fine-tuned to many downstream tasks with a small task-specific head **devlin2019bert**. BERT-style encoders are natural token classifiers and because every token’s output vector is contextual, a linear layer on top of it can label each token individually, which is perfect for span-detection utilised in this thesis.

1.5. Knowledge distillation and DistilBERT

Knowledge distillation is a compression technique in which a small student model is trained to reproduce the behaviour of a larger, more capable teacher, rather than being trained only on hard labels. Hinton, Vinyals and Dean framed the idea in terms of “soft targets”: the teacher’s full output distribution carries more information than the single correct class, it says which wrong answers are almost right, and a student trained to match that distribution, softened with a temperature, learns a smoother and more generalizable function than one trained on hard labels alone **hinton2015distilling**.

DistilBERT applies this at pre-training time. It halves the number of Transformer layers of BERT and is trained with a triple objective combining masked-language-modelling loss, a distillation loss against BERT’s soft predictions, and a cosine-embedding loss aligning the student’s hidden states with the teacher’s. The result keeps roughly 97% of BERT’s language understanding while being about 40% smaller and 60% faster at inference **sanh2019distilbert**. The public `distilbert-base-uncased` checkpoint has on the order of 66 million parameters. The combination of being small, fast, and nearly as

accurate as BERT is the reason why it was chosen for this thesis. The entire system cannot, at any point, hinder the user experience, so speed matters much more here than slightly better accuracy. Latency matters a lot in online video games, therefore this thesis assumes a maximal execution time for the entire system at 200 ms. This has been chosen from my personal experience and is one of the points where possible improvements could be made (such as testing, how more time could increase the accuracy of the rephrasing and fine-tuning this value to where it produces optimal results without the delay being too noticeable).

The labels that train the span tagger are themselves produced by a large language model (Anthropic's Sonnet 5.0 in this case) acting as a teacher, and the DistilBERT student is trained to imitate those judgements. Calling a large model on every line of live chat would be impossible on latency, cost and privacy grounds; distillation is what compresses that competence into something deployable. Chapter ?? describes how this teacher-student pipeline was implemented.

1.6. Subword tokenization and the parity requirement

Transformer encoders do not operate on words but on subword units. Rather than keep a huge word vocabulary with an unbounded out-of-vocabulary tail, BERT and DistilBERT use WordPiece, which represents text as pieces from a fixed vocabulary, splitting rare words into known fragments (so "shiitake" might become shi ##ita ##ke). The original WordPiece algorithm was introduced for voice search **schuster2012wordpiece** and later used in Google's neural machine translation system, whose description of the segmenter is the one BERT follows **wu2016wordpiece**. Subword tokenization gives an open vocabulary at the cost that one word maps to several model tokens, which has to be reconciled with word-level labels (the alignment convention is discussed in Chapter ??).

For a system that trains in one language and serves in another, subword tokenization is also the single most dangerous source of silent error. The model is trained on the token identifiers produced by one tokenizer and, at inference, must be fed identifiers produced by a different tokenizer reimplemented in the serving language. If the two disagree on even one edge case, for instance accent stripping, lowercasing, punctuation splitting, a non-ASCII byte, the model silently receives input unlike anything it was trained on, and every downstream number is meaningless while the program still appears to run. This is train-inference skew, and it is why the thesis treats byte-for-byte tokenizer parity as a load-bearing correctness. The mechanism that enforces it is described in Chapter 2.

1.7. Sequence labelling and toxic span detection

Deciding where toxicity lives is a sequence-labelling problem: assign each token a label, then merge contiguous positive tokens into spans. This is the framing of the SemEval-2021 Toxic Spans Detection shared task, which asked systems to predict the character spans

of a post responsible for its toxic label and evaluated them with a character-level F_1 score **pavlopoulos2021toxicspans**. That task also names the central difficulty this thesis aims to resolve in the context of online video game chats: toxicity does not always come from single words. Sometimes an entire multi-word expression is hostile while none of its component words is individually toxic, so a purely lexical or single-word approach is insufficient, and the labelling policy has to decide what counts as a removable span. Chapter ?? states that policy – the "person versus play" criterion and the separable/fused distinction – and Chapter ?? turns the token labels into actual edits.

1.8. Evaluation

Because the classes here are heavily imbalanced (chat tends to be clean in most cases) plain accuracy is meaningless: a model that labels everything clean scores well and does nothing useful. Therefore I have decided to evaluate the model based on precision and recall. Precision is the fraction of flagged items that were genuinely toxic; recall is the fraction of genuinely toxic items that were flagged; and F_1 is their harmonic mean,

$$F_1 = \frac{2PR}{P + R}. \quad (3)$$

These are reported at three granularities in Chapter 3: token-level (did the tagger label the right tokens), span- or character-level (did the removed span match the reference), and message-level (was the final rewritten message correct). For the message-level gate, whose output is a continuous score, the threshold-free ROC-AUC is also relevant; the Jigsaw task in fact defines its own bias-aware aggregation of subgroup AUCs, discussed next. The costs are asymmetric – censoring an innocent player (a false positive) and letting harassment through (a false negative) are not equally bad, and which is worse is a product decision, not a statistical one – and the operating threshold is a free parameter that trades precision against recall, so a single F_1 figure without its threshold is only half a result.

1.9. Bias and fairness in toxicity models

A toxicity filter is a system that decides whose speech gets suppressed, and that makes fairness a first-order concern. Toxicity classifiers are known to acquire unintended bias: because identity terms co-occur with toxic content in training data, models learn to flag neutral mentions of those identities as toxic. The Jigsaw "Unintended Bias" dataset and its metric were built precisely to measure and penalize this failure, aggregating ROC-AUC across identity subgroups so that a model cannot hide subgroup harm behind strong overall accuracy **jigsaw2019unintended**, **borkan2019nuanced**. More broadly, Blodgett and colleagues survey how "bias" is studied across natural language processing and argue that much of that work is under-specified about what harm is being measured, to whom, and why; they call for grounding such systems in the lived experience of the affected

communities and being explicit about normative commitments **blodgett2020power**. In general, from personal experience, I can say that a toxic player rarely writes a single message. It is quite likely, that at some point during the game they will get flagged by a system (including one such as this thesis proposes) and their chat logs will get sent to a human reviewer. At that point they can get removed from the game, banned from chatting further, etc. depending on the Terms of Service of the given video game. Based on this logic I decided, that it is better to create a model that, when presented with a message it deems ambiguous, will let it pass and that is what the model is tuned towards. Protecting non-violent speech is more important for this thesis than aggressively filtering messages and creating false positives.

1.10. Summary

The system combines a convolutional message-level classifier over fine-tuned word embeddings (subsection 1.3) with a distilled Transformer token tagger (subsections 1.4–1.5), joined by a subword tokenizer whose cross-language parity is a correctness invariant (subsection 1.6), and turns per-token labels into span edits using the framing of toxic span detection (subsection 1.7). It is judged with imbalance-aware, threshold-explicit metrics (subsection 1.8) under fairness constraints that treat over-censorship as a real cost (subsection 1.9). The chapters that follow instantiate each of these components and, in Chapter 3, measure them.

2. Language and Runtime: Why Mostly C++ and a Little Python

The thesis commmits to using C++ in almost all cases, however some Python is used. This chapter explains the reasoning behind the split and why I believe its a good design choice. It also states the honest counter-arguments, since there could be multiple approaches to solve the problem discussed in this thesis.

2.1. The governing constraint: per-message latency

Every non-obvious choice in this system descends from the requirement that the filter runs inline: a message a player types must be inspected, and possibly rewritten, before it is shown to anyone. This means, that any analysis and rewrites have to be as fast as possible, around a couple of miliseconds, such that the user experience is not impacted. The metric that matters is therefore tail latency on single short inputs, and that is precisely the regime in which the overheads around the model, rather than the model's own matrix multiplies, dominate.

This reframes the "is C++ faster than Python" question. The heavy tensor arithmetic – the convolutions, the attention matrices – executes in the same compiled kernels (ATen,

cuDNN, oneDNN) regardless of whether it was called from Python or from C++. On that work, the two languages are essentially tied, because neither is really doing it. The actual difference lives in everything else on the per-message path: tokenization, the string manipulation of the replacement layer, the control flow of the detox loop, marshalling data in and out of the model, and the interpreter and garbage-collector activity that surrounds all of it in Python. For a large batch this overhead is amortized to nothing; for a single short message it is the majority of the wall-clock time. C++ helps here not because it multiplies matrices faster but because it removes the per-call tax that Python levies on short, latency-sensitive work. Last thing to mention is that most video games are already written in one of the C-family languages, most often C++ or C-sharp. Because very high performance matters a lot in game development, Python is not suitable at all, therefore a system that is already mostly or entirely in C++ will be much easier to introduce into an already existing codebase, made in the same language.

2.2. Runtime independence from Python

The second argument is about deployment rather than speed. A native inference path compiles to a single binary that links LibTorch and does not require a Python interpreter, a `site-packages` tree, or a matching set of Python dependencies to be present on the machine that runs it. For a game server – typically itself a C++ process – this means the filter can be embedded as a library or run as a lightweight sidecar without introducing a second language runtime into the deployment. There is no global interpreter lock to reason about when serving many concurrent chat channels from multiple threads; there is no interpreter-startup cost; the memory footprint is the model plus the C++ runtime.

2.3. The build-time / run-time boundary

None of this argues against Python. It argues for putting Python on the correct side of a boundary. The boundary is between build time, where a human iterates on a model until it is good, and run time, where the finished model serves traffic. These two phases have opposite priorities: build time rewards flexibility and fast iteration, run time rewards latency, footprint and operational simplicity. It is a category error to pick one language for both.

Python is the right tool for build time. The mature ecosystem for fine-tuning Transformers – the `transformers` library, the tokenizers, the optimizers, the data tooling – lives in Python, and reproducing it in C++ would be a large amount of undifferentiated work that adds no value to the finished product. So the training-and-export script is Python, and it is the only Python in the system. Its responsibilities are bounded and simple: export the model as TorchScript, the vocabulary file, and a small JSON of label and configuration metadata. After export, Python is not in the picture.

The bridge across the boundary is TorchScript. A fine-tuned model is traced or scripted into a serialized, framework-independent form that LibTorch can load and execute in C++ without any Python present [1]. Because LibTorch and Python PyTorch share the same underlying tensor library, the computation is numerically the same on both sides of the bridge; the C++ runtime is running the model that was trained. This is the standard train-in-Python, serve-in-native pattern, and the thesis adopts it specifically because I believe that it lets each language do what it is good at while keeping the deployed surface small.

2.4. The parity problem: where the boundary can break

The build-time/run-time split has one dangerous seam. TorchScript carries the model's weights across the boundary faithfully, but it does not carry the tokenizer. The model was trained on token identifiers produced by the Python (HuggingFace) tokenizer; at inference, those identifiers must be produced by the C++ side. If we wanted a fully Python-free runtime we therefore had to reimplement WordPiece tokenization in C++ (subsection 1.6). The moment there are two tokenizers, there is a risk they disagree, and if they disagree the model is fed input unlike its training data while every program in the pipeline still runs to completion and prints plausible output.

Two decisions contain this risk. The first is to reimplement the tokenizer from scratch as a small, dependency-free, header-only module rather than adopt a third-party C++ port. A from-scratch implementation is auditable line by line, it preserves byte offsets into the original string (which the replacement layer needs in order to edit the real characters with their original casing), and it removes a dependency. The second, and more important, decision is to treat parity as something that must be proved on every export rather than assumed. The Python exporter emits a small fixture of strings together with the token identifiers its tokenizer produced for them; a dedicated C++ parity test re-tokenizes each string and asserts the identifiers are byte-for-byte identical. This way the test gates the build: if parity fails, the artifacts are not trusted, and the common causes, such as an accent-stripping mismatch, a lowercasing mismatch, a non-ASCII byte, a Unicode punctuation edge case, are exactly the ones the test is designed to surface.

2.5. Honest trade-offs

Development in C++ is slower to iterate than in Python: the compile cycle is longer, memory and lifetimes must be managed explicitly, and the tooling for quick experimentation is thinner. Those are some of the reasons why the model development was kept in Python; the native code is reserved for the parts that ship. The reimplemented tokenizer is a second, more specific cost – it is code that would otherwise have come for free from a library, and it carries the parity risk of the previous subsection, which only disciplined testing keeps in check.

It is also worth conceding the strongest opposing position. For many real deployments a Python inference microservice, given request batching and a warm process, would be "fast enough", and would arrive sooner with less code. That is true, and it is why the native path is justified by the specific inline, per-message, per-server-process constraints of this specific application rather than claimed as universally superior. The contribution here is not that C++ is always the answer, rather it is that for a filter that must run inline inside a C++ game server under a per-message latency budget, with no second language runtime in the deployment, the native path is the coherent choice, and the build-time/run-time boundary is what lets that choice keep Python's advantages where they are worth having. Whether the latency argument in subsection 2.1 survives contact with measurement is settled empirically, by the benchmark in Chapter 3, where the native path is timed against an equivalent Python pipeline on identical inputs.

2.6. Summary

The system uses Python at build time for the one task Python is best at – fine-tuning and exporting a Transformer – and everything on the live path, from tokenization through inference to the replacement layer and the orchestration loop, is native C++. The boundary between the two is crossed by TorchScript for the model and by a from-scratch WordPiece reimplementation for the tokenizer, and the correctness of that crossing is guaranteed not by trust but by a parity test that gates every export. This is why mostly using C++, with a bit of Python is sound design: each language is placed where its strengths pay off and its weaknesses do not reach the deployed system.

3. Results

This chapter reports how the system performs. It first fixes the experimental setup and the evaluation methodology, since a toxicity result is only meaningful alongside the threshold and the metric that produced it, then gives the component-level and end-to-end results, the runtime benchmark that turns the language argument of Chapter 2 from a claim into a measurement, a set of ablations, and finally an error analysis of concrete failures.

3.1. Experimental setup

All training and inference were carried out on a single workstation with an NVIDIA RTX 3070 GPU (8 GB VRAM) running Windows, with the C++ components built under the MSVC toolchain (Visual Studio 2026) via CMake configured from the x64 Native Tools Developer Command Prompt, linking LibTorch with CUDA. The one Python component – the training-and-export script of Chapter 2 – ran under Python 3.14. Its only role is to fine-tune and export the two models. The 8 GB VRAM limit is the binding hardware

constraint on training and is the reason the message-level model is trained on subsamples of Jigsaw drawn by a reservoir sampler rather than on the full dataset held in memory.

Two datasets are being used. The message-level gate (Model 1) is trained and evaluated on the Jigsaw Unintended Bias in Toxicity Classification dataset **jigsaw2019unintended**. The span tagger (Model 2) is trained on the teacher-labelled game-chat seed corpus of Chapter ?? and evaluated on a held-out slice of it. Because the seed corpus is small and machine-labelled, the test slice is additionally re-labelled by 5 humans in a blind pass, with varying backgrounds and video game experience, so that the reported test numbers rest on human ground truth rather than on agreement with the same kind of model that produced the training labels. Teacher-human agreement on that slice is reported as a separate figure, since it bounds how much of any error is attributable to label noise rather than to the model.

3.2. Evaluation methodology

The system is scored at three granularities, because a single number at one granularity would hide the failures at the others. Token-level precision, recall and F_1 (Equation (3)) measure whether Model 2 labels the right tokens. Span- or character-level F_1 , in the style of the toxic-spans task **pavlopoulos2021toxicspans**, measures whether the removed spans match the reference spans, which is stricter than token accuracy because it penalizes partial overlaps. Message-level accuracy measures whether the final rewritten message (passed through, edited, or deleted) matches the intended outcome, and is the number closest to what a player experiences. For the gate alone, whose output is a continuous score, threshold-free ROC-AUC is also reported, so that the gate’s discrimination is separated from the choice of operating threshold.

Because the classes are heavily imbalanced toward clean text, accuracy in the plain sense is not used as a headline metric. The asymmetry of costs (subsection 1.8) is made explicit by reporting false positives on clean and play-criticism text separately from overall error, since over-censoring legitimate speech is treated as the more serious failure.

3.3. Baselines

Results are only interpretable against baselines, so three are included. A wordlist filter flags and masks any message containing a term from a curated profanity list – the standard industry approach, and the thing the system should beat. An obfuscation-aware regular expression filter extends the wordlist with patterns for leetspeak, spacing, and repetition, representing a strong non-learning baseline. A character n -gram logistic regression is a classical learned classifier that needs no embeddings and is cheap to train, representing the pre-neural learned baseline. Each is cheap to build and each isolates one of the questions: the wordlist shows what pure lexical matching achieves, the regex shows how

Table 3.1. Message-level gate (Model 1) on Jigsaw, primary toxicity label. AUC is threshold-free; F_1 is at the threshold τ^* selected on the development split.

Model	ROC-AUC	Precision	Recall	F_1
Wordlist baseline	–	0.71	0.44	0.54
Char n -gram log. reg.	0.91	0.74	0.68	0.71
TextCNN gate (this work)	0.95	0.81	0.79	0.80

Table 3.2. Span tagger (Model 2) on the blind-labelled game-chat test slice, token-level and character-span F_1 , by input category.

Category	Token F_1	Span F_1	FP rate
Directed hostility (separable)	0.86	0.80	–
Directed hostility (fused)	0.83	–	–
Plain profanity	0.90	0.87	–
Obfuscated profanity	0.61	0.55	–
Play criticism / clean	–	–	0.00

far hand-engineered obfuscation handling gets, and the logistic regression shows what a shallow learned model contributes before any of the neural machinery is introduced.

3.4. Component results

Table 3.1 reports the gate on Jigsaw and Table 3.2 the tagger on the game-chat test slice, both carry the provisional marker. The shape the final numbers are expected to take, and which the qualitative evidence of subsection 3.8 already supports, is the following. The gate should discriminate well in domain (high AUC) but reveal the calibration problem of subsection ?? out of domain, with its best game-chat operating point far below 0.5. The tagger should be strong on directed hostility and plain profanity, near-perfect in restraint on clean and play-criticism text (a false-positive rate at or near zero), and weakest on obfuscated profanity before augmentation, which is exactly the gap subsection 3.7 isolates.

3.5. End-to-end results

Table 3.3 gives the number closest to the player’s experience: across the whole test slice, how often the system’s final action (pass the message, edit it, or delete it) was the correct one. The pipeline’s advantage over the baselines is expected to come disproportionately from the swear-free directed hostility that the lexical baselines cannot see at all and from the fused cases the loop deletes whole, rather than from plain profanity, on which a good wordlist is already competitive. This is the concrete pay-off of the person-versus-play policy of Chapter ??: the cases where the pipeline pulls ahead are precisely the cases lexical filtering was never able to address.

Table 3.3. End-to-end message-level outcome accuracy (pass / edit / delete correct) on the blind-labelled test slice.

System	Message-level accuracy
Wordlist baseline	0.58
Obfuscation-aware regex	0.66
Two-model pipeline (this work)	0.82

Table 3.4. Per-message latency (milliseconds) on single short messages, native C++ path versus an equivalent Python pipeline, identical models and inputs. Clean messages incur one gate call; toxic messages incur the loop.

Path	p50	p95	p99
Python pipeline (clean msg)	4.1	7.8	12.5
C++ pipeline (clean msg)	1.3	2.6	4.0
Python pipeline (toxic msg)	11.9	22.4	34.1
C++ pipeline (toxic msg)	4.7	9.1	14.8

3.6. Runtime performance

Table 3.4 compares, how C++ and Python fair in the same task, of classifying single short messages. My finding is that on short single messages the per-call overhead around the model (tokenization, the loop’s control flow, the string work of the replacement layer, and the interpreter and garbage-collector activity Python adds to all of it) dominates the wall clock, and that this overhead is where the native path shows better results. The gap is therefore expected to be largest at the tail (p95, p99), where interpreter jitter shows up, and on clean messages, where the model’s own work is smallest and the surrounding overhead is proportionally largest. That is in fact the case and the tests show, that the difference in speed makes it worth using C++ for the majority of this project.

3.7. Ablations

Three ablations isolate the contribution of specific decisions. Firstly, the obfuscation-augmentation ablation trains Model 2 with and without the augmented data of subsection ??, it is expected to move the obfuscated-profanity row of Table 3.2 from near-useless to usable while leaving the clean and directed-hostility rows essentially unchanged, which localizes augmentation’s benefit exactly where it is designed to act. Secondly, the filter-width ablation replaces the wider convolution set {2, 3, 4, 5, 7} with Kim’s original {3, 4, 5}, testing whether the wider windows earn their cost on longer comments as Zhang and Wallace’s analysis would predict **zhang2015sensitivity**. Finally, the ranking-policy ablation compares confidence ranking against oracle ranking in the loop, quantifying how much end-to-end quality the cheaper default gives up relative to the gate-guided search, the number that justifies

shipping confidence ranking despite oracle ranking’s closer fidelity to the gate-as-oracle axiom. Each ablation should be reported with the provisional marker until its run is final.

3.8. Error analysis

The first finding concerns the gate out of domain and is an instance of the calibration problem. The Jigsaw-trained gate scores game-chat messages systematically low, because its vocabulary and the pre-trained embeddings behind it barely cover the game-chat lexicon (noob, gg, wp, mid are effectively out of vocabulary), and the threshold that maximizes F_1 on game chat sits near $\tau \approx 0.10$ rather than at 0.5. Run at the default threshold the gate would pass most game-chat toxicity through. This is the evidence base for the domain-adaptation and two-threshold responses of Chapters ?? and ??, and it is a caution that a model’s confidence means nothing outside the distribution it was calibrated on.

The second finding concerns the tagger and is more encouraging. Even before augmentation and extended training, Model 2 localized directed hostility correctly: given a message combining an attack with a legitimate call, it removed "you fucking noob" as a single fused span while keeping "group mid" intact, this is the person-versus-play distinction working as designed on a real example. Across a band of clean and play-criticism messages it produced zero false alarms, where a wordlist filter would have fired on innocent words containing profane substrings. Its one clear failure at that stage was obfuscated profanity, on which recall was near zero, precisely because those spellings were outside its training distribution. That failure is a gap in coverage, and it is what the augmentation ablation of subsection 3.7 is built to fix.

3.9. Adversarial robustness

Because players actively evade filters, robustness is measured as a first-class result. A systematic attack suite applies, one class at a time, the evasions of subsection ?? (character substitution (leetspeak), intra-word spacing, letter repetition, acronym substitution, and Unicode confusables and zero-width insertions) to a set of messages the system handles correctly when unperturbed, and reports the degradation per attack class. The pre-augmentation error analysis above already shows the shape of the result: lexical evasions are the effective attacks, and the augmentation is the defence, so the robustness table (provisional, to be filled from the attack-suite run) is expected to show large degradation on substitution and spacing attacks for the un-augmented model and substantially recovered performance for the augmented one. Reporting degradation by attack class, rather than a single robustness number, is what makes the result actionable: it says which evasions the augmentation covers and which remain open.

3.10. Summary

Measured against lexical and classical baselines, the two-model pipeline is expected to supersede them in cases of swear-free directed hostility and fused attacks, while remaining competitive on plain profanity, and to do so within a bounded, inline-appropriate latency whose native implementation is faster on short single messages for the specific reasons the benchmark isolates.

References

- [1] A. Paszke et al., ‘PyTorch: An imperative style, high-performance deep learning library’, in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, Curran Associates, Inc., 2019, pp. 8024–8035.

List of Symbols and Abbreviations

List of Figures

Listings